

Shortest Cycle Approach

This is **LeetCode 2608 — Shortest Cycle in a Graph**.

The key idea is: **a cycle can be found by temporarily removing one edge and checking whether its two endpoints are still connected**. That gives a brute-force solution. But there is a much better BFS-based approach.

1. First understand what we need

Suppose we have:



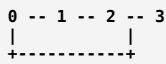
There is a cycle:

```
0 → 1 → 2 → 0
```

Its length is 3.

We need the **smallest number of edges in any cycle**.

For example:



The cycle length is 4.

2. Brute Force Approach

A very direct thought is:

For every edge (u, v) , remove that edge and check whether u and v can still reach each other.

Why does this work?

Suppose:



If we remove the direct edge:

```
u ----- v
```

but there is still another path:

```
u → a → b → v
```

then adding the removed edge gives:

```
u → a → b → v → u
```

which is a cycle.

So:

For every edge (u, v) :

1. Temporarily ignore (u, v) .
2. Run BFS/DFS from u .
3. Check whether we can reach v .
4. If yes:

```
cycle length = distance(u, v) + 1
```

5. Take the minimum.

Example

```
0 -- 1
|    |
2 -- 3
```

Take edge:

```
0 -- 1
```

Remove it.

Now:

```
0
|
2 -- 3
    |
    1
```

Shortest path from 0 to 1:

```
0 → 2 → 3 → 1
```

Distance = 3.

Therefore cycle:

```
0 → 2 → 3 → 1 → 0
```

Length:

```
3 + 1 = 4
```

3. Brute Force Code

```
C++
class Solution {
public:
    int findShortestCycle(int n, vector<vector<int>>& edges) {
        int ans = INT_MAX;

        for (auto &edge : edges) {
            int u = edge[0];
            int v = edge[1];

            // BFS after removing edge (u, v)
            vector<int> dist(n, -1);
            queue<int> q;

            dist[u] = 0;
            q.push(u);

            while (!q.empty()) {
                int node = q.front();
                q.pop();

                for (auto &e : edges) {
                    // Ignore the current edge
                    if ((e[0] == u && e[1] == v) ||
                        (e[0] == v && e[1] == u)) {
                        continue;
                    }

                    int a = e[0];
                    int b = e[1];

                    int next = -1;

                    if (a == node)
                        next = b;
                    else if (b == node)
                        next = a;

                    if (next != -1 && dist[next] == -1) {
                        dist[next] = dist[node] + 1;
                        q.push(next);
                    }
                }
            }

            // u can reach v after removing edge
            if (dist[v] != -1) {
                ans = min(ans, dist[v] + 1);
            }
        }

        return ans == INT_MAX ? -1 : ans;
    }
};
```

```
}  
};
```

But there is a problem

This implementation is quite inefficient because for every BFS node we scan **all edges**.

We can improve the brute force using an adjacency list.

Then:

- For each edge $\rightarrow m$ possibilities
- BFS $\rightarrow O(n + m)$

So:

```
O(m * (n + m))
```

Given:

```
n <= 1000  
m <= 1000
```

this is actually manageable.

But we can do even better conceptually.

4. Optimal Approach — BFS from Every Node

The most standard optimal solution is:

Run BFS starting from every vertex and detect a cycle while doing BFS.

The important observation is this:

Suppose BFS starts from node 0.



BFS distances:

```
dist[0] = 0  
dist[1] = 1  
dist[2] = 1  
dist[3] = 2
```

When processing node 1, we see neighbor 3.

But:

```
dist[3] != -1
```

So 3 has already been visited.

However, we cannot simply say:

"Visited node means cycle."

Because in an undirected graph, we always see the edge back to our parent.

For example:

```
0 -- 1
```

While processing 1, we see 0.

0 is already visited, but this is **not a cycle**.

Therefore we need to remember the **parent**.

5. The Critical BFS Logic

For every node:

```
C++
```

```
for (int neighbor : graph[node]) {
    if (dist[neighbor] == -1) {
        // First time visiting neighbor
    }
    else if (neighbor != parent[node]) {
        // We found a cycle
    }
}
```

Why?

Case 1 — Unvisited neighbor

```
dist[neighbor] == -1
```

We have discovered a new node.

So:

```
C++
dist[neighbor] = dist[node] + 1;
parent[neighbor] = node;
```

Case 2 — Already visited neighbor

If:

```
C++
neighbor == parent[node]
```

ignore it.

That's just the normal undirected edge:

```
parent
|
node
```

But if:

```
C++
neighbor != parent[node]
```

then we found another connection to an already visited node.

That means:

```
node ----- neighbor
 \         /
  existing
   path
```

Therefore, there is a cycle.

6. How do we calculate the cycle length?

Suppose BFS gives:

```
dist[node] = 3
dist[neighbor] = 2
```

There is an edge:

```
node ---- neighbor
```

The cycle consists of:

```
path from start → node
+
path from start → neighbor
+
edge node-neighbor
```

Therefore:

```
cycle length
= dist[node] + dist[neighbor] + 1
```

So:

```
C++
int cycleLength = dist[node] + dist[neighbor] + 1;
```

7. Why BFS gives the shortest cycle

This is the important part.

We perform BFS from **every vertex**.

BFS guarantees:

The first time we reach a node, we reached it using the shortest possible path from the starting node.

Suppose a cycle is:



If BFS starts from A, it explores the graph level by level.

When it finds an edge connecting two already-reached branches, it gets a cycle candidate.

Since BFS uses shortest distances:

```
dist[u] + dist[v] + 1
```

gives the shortest cycle involving that edge from that BFS source.

And because we run BFS from **every node**, we don't miss the globally shortest cycle.

8. Optimal Code

```
C++
class Solution {
public:
    int findShortestCycle(int n, vector<vector<int>>& edges) {
        // Build adjacency list
        vector<vector<int>> graph(n);

        for (auto &edge : edges) {
            int u = edge[0];
            int v = edge[1];

            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        int ans = INT_MAX;

        // BFS from every node
        for (int start = 0; start < n; start++) {
            vector<int> dist(n, -1);
            vector<int> parent(n, -1);

            queue<int> q;

            dist[start] = 0;
            q.push(start);

            while (!q.empty()) {
                int node = q.front();
                q.pop();

                for (int neighbor : graph[node]) {
                    // First time visiting this node
                    if (dist[neighbor] == -1) {
                        dist[neighbor] = dist[node] + 1;
                        parent[neighbor] = node;
                        q.push(neighbor);
                    }
                    // Already visited and not our parent
                    else if (neighbor != parent[node]) {
                        int cycleLength =
                            dist[node] + dist[neighbor] + 1;
                        ans = min(ans, cycleLength);
                    }
                }
            }
        }
    }
};
```

```

    }
    return ans == INT_MAX ? -1 : ans;
}
};

```

9. Dry Run

Take:

```

n = 7
edges:
0--1
1--2
2--0
3--4
4--5
5--6
6--3

```

There are two cycles:

```
0 → 1 → 2 → 0
```

length 3.

and

```
3 → 4 → 5 → 6 → 3
```

length 4.

So answer is:

```
3
```

BFS from 0

Initially:

```

dist:
0 = 0
1 = -1
2 = -1

```

Process 0.

Neighbors:

```

1
2

```

Set:

```

dist[1] = 1
dist[2] = 1

```

Now:

```

  0
 / \
1   2

```

Process 1.

Neighbors:

```

0
2

```

0 is parent → ignore.

2 is already visited and is **not** parent.

Therefore:

```

cycleLength
= dist[1] + dist[2] + 1
= 1 + 1 + 1
= 3

```

So:

```
ans = 3
```

10. Complexity

Let:

```
n = number of vertices
m = number of edges
```

Building adjacency list:

```
O(n + m)
```

We perform BFS from every vertex.

One BFS:

```
O(n + m)
```

There are n BFS runs:

```
O(n(n + m))
```

So:

Time

```
O(n * (n + m))
```

Space

```
O(n + m)
```

For this problem:

```
n <= 1000
m <= 1000
```

so this is completely fine.

11. Brute Force vs Optimal

Approach	Idea	Time
Brute force	Remove every edge + BFS/DFS	$O(m(n+m))$
Optimal	BFS from every vertex	$O(n(n+m))$

With this problem's constraints, **both pass**, but the BFS-from-every-node solution is the cleaner standard solution.

The intuition to remember

Think of the optimal solution like this:

Start BFS from every node. Whenever I find an edge connecting two already-visited nodes that aren't parent-child, that edge closes a cycle. Because BFS gives shortest distances, $\text{dist}[u] + \text{dist}[v] + 1$ gives the cycle length. Take the minimum over all BFS runs.

The **parent check** is the most important part:

```
C++
else if (neighbor != parent[node])
```

because in an undirected graph, seeing your parent again is normal and **doesn't mean there is a cycle**.

ChatGPT can make mistakes. Check important info.